

DATA 260 Homework 4

Section 0. Personal Configuration and Domain

SID4	9486
PORT_BASE	8486
PREFIX	s9486
SEED	9486
VERIFY_SEED	269486
DOMAIN_ID	6 - Rental housing listings
Hardware	Apple Mac with M1 chip and 8 GB unified memory
Local model	llama3.2:3b (via Ollama)
Embedding model	sentence-transformers/all-MiniLM-L6-v2
Database	MySQL 9.4.0, database s9486_rel
GitHub repository	https://github.com/Kavan1804/data260-9486
Content-complete source commit	f0e5612234610707dafe13427b12c748202dd070
Final submission tag	hw4

HW1-HW3 used qwen3:4b. For HW4 the installed qwen3:4b build kept writing its reasoning into the answer text even with think=false, so answers were cut off before the actual answer (see AI_USE.md). Part 4 therefore uses the smaller non-thinking llama3.2:3b, which fits the 8 GB M1. The SQLAlchemy session factory is named SessionLocal after the TA confirmed that the name in the handout (db_session_basede26) was a typo.

Reproducible commands (run from Homework4/)

```
python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
cp .env.example .env # DATABASE_URL=mysql+pymysql://USER:PASSWORD@localhost:3306/s9486_rel
make init-db && make create-user
make backend # FastAPI on http://localhost:8486
cd frontend && npm install && npm run dev # React on http://localhost:5173
make seed && make explain && HW4_EMAIL=<email> make measure
ollama pull llama3.2:3b && make rag
make verify-hw04 && make report
```

Repository access: both collaborators (Sbnikitha, supriyaselvangesan) are given access to <https://github.com/Kavan1804/data260-9486>.

Project Folder Structure

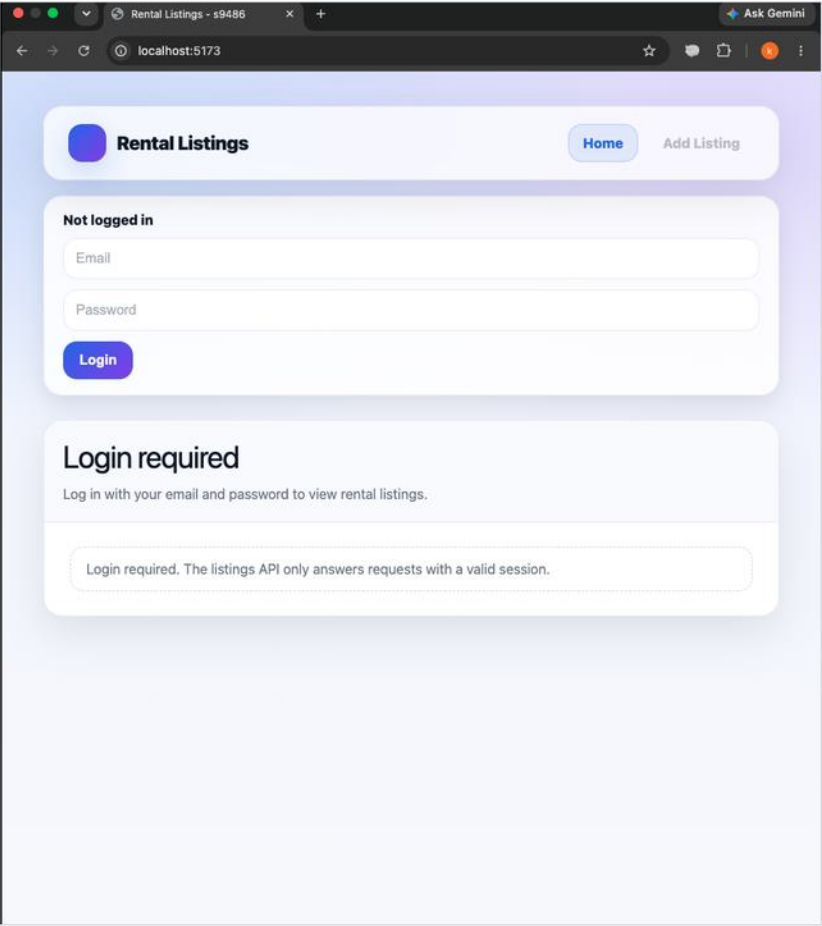
HW4 continues the per-homework layout used for HW1-HW3 inside the same data260-9486 repository. The professor's DATA236_demo4 starter was integrated rather than copied: its backend file layout (database.py, models.py, crud.py, session_crud.py, schema.py, main.py) became backend/app/, and its React client (Vite, react-router-dom, axios, Navbar, login bar, styles.css) became frontend/. The starter folder itself is gitignored because it contains its own .venv, node_modules and .env files.

Folder tree generated from the repository

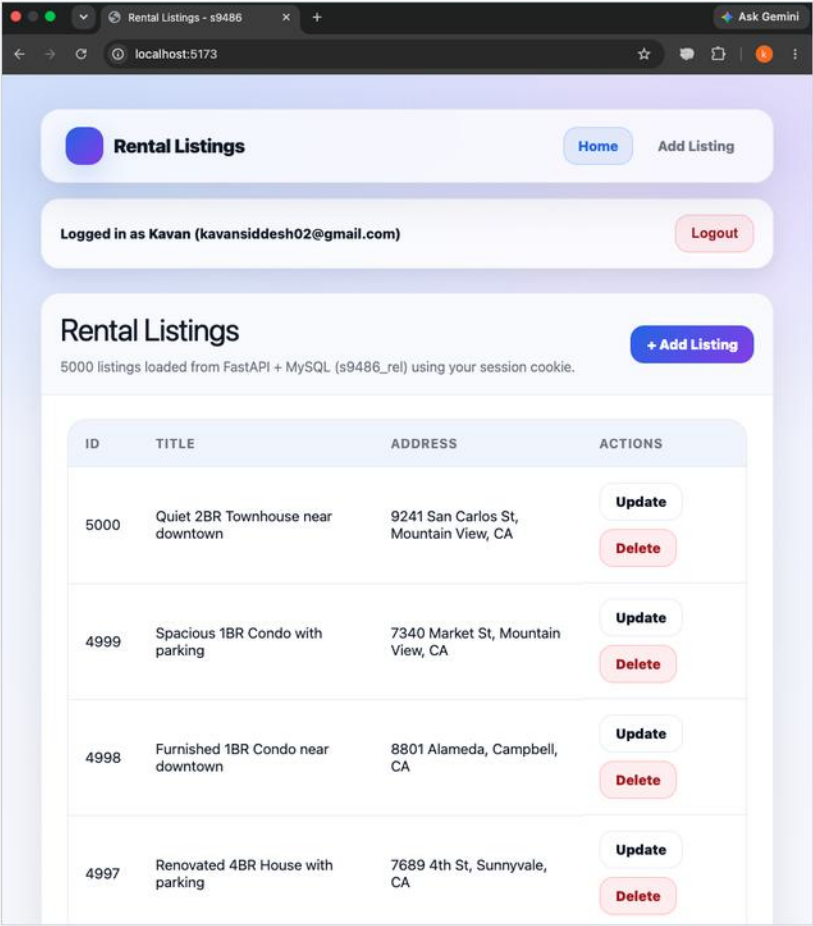
```
Homework4/
|-- backend/
|   |-- app/
|   |   |-- __init__.py
|   |   |-- crud.py
|   |   |-- database.py
|   |   |-- main.py
|   |   |-- models.py
|   |   |-- query_counter.py
|   |   |-- schema.py
|   |   |-- session_crud.py
|   |-- __init__.py
|-- corpus/
|   |-- SOURCES.md
|   |-- ca_civil_code_1941_1_habitability.txt
|   |-- ca_civil_code_1950_5_security_deposit.txt
|   |-- ca_tenant_notice_requirements.txt
|   |-- ca_tenant_protection_act_ab1482.txt
|   |-- fair_housing_act.txt
|-- frontend/
|   |-- src/
|   |   |-- api/
|   |   |   |-- listingsApi.js
|   |   |-- components/
|   |   |   |-- Navbar.jsx
|   |   |-- pages/
|   |   |   |-- CreateRecord.jsx
|   |   |   |-- DeleteRecord.jsx
|   |   |   |-- Home.jsx
|   |   |   |-- Login.jsx
|   |   |   |-- UpdateRecord.jsx
|   |   |-- app.jsx
|   |   |-- main.jsx
|   |   |-- styles.css
|   |-- .env.example
|   |-- index.html
|   |-- package-lock.json
|   |-- package.json
|   |-- vite.config.js
|-- reports/
|   |-- hw04/
|   |   |-- raw/
|   |   |   |-- basic_rag.jsonl
|   |   |   |-- context_rag.jsonl
|   |   |   |-- evaluation.csv
|   |   |   |-- evaluation_table.md
|   |   |   |-- ... 12 more files
|   |   |-- screenshots/
|   |   |   |-- mysql_count_listings.png
|   |   |   |-- mysql_describe_listing_inquiries.png
|   |   |   |-- mysql_describe_listings.png
|   |   |   |-- mysql_describe_sessions.png
|   |   |   |-- ... 37 more files
|   |   |-- AI_USE.md
|   |   |-- METRICS.md
|   |   |-- RAG_ANALYSIS.md
|   |   |-- RUN_LOG.txt
|   |   |-- final_report.pdf
|-- scripts/
|   |-- explain_index.py
|   |-- init_db.py
|   |-- measure_n1.py
|   |-- render_report.py
|   |-- seed_n1.py
|   |-- verify_hw04.py
|-- .env.example
|-- .gitignore
|-- Makefile
|-- README.md
|-- config.py
|-- rag.py
|-- rag_questions.json
|-- requirements.txt
|-- verification.json
```

Part 1. React Client - Login (Login.jsx) and Home Page (Home.jsx, route /)

App checks the HTTP-only session cookie on load by calling GET /auth/me (the cookie itself cannot be read by JavaScript). Login.jsx posts email + password; the backend sets the cookie. While logged out, Home shows "Login required", the Add Listing link is disabled, and /create, /update/:id and /delete/:id render a Login required card through RequireAuth.



Logged out: 'Login required' and a disabled Add Listing link



After login with email + password: Home (/) lists all 5,000 listings from MySQL

Part 1. React Client - II. Add a New Record (CreateRecord.jsx, route /create)

CreateRecord receives onAdd as a prop, keeps the two inputs in useState, and App redirects to / after the API returns the new auto-incremented ID.

Rental Listings

[Home](#)[Add Listing](#)

Logged in as Kavan (kavansiddesh02@gmail.com)

Logout

Add Listing

Enter the listing title and property address

Listing title

Home

Property address

2779 castor drive

Add Listing

Add Listing form (title = primary field, address = secondary field)

Rental Listings

[Home](#)[Add Listing](#)

Logged in as Kavan (kavansiddesh02@gmail.com)

Logout

Rental Listings

5001 listings loaded from FastAPI + MySQL (s9486_rel) using your session cookie.

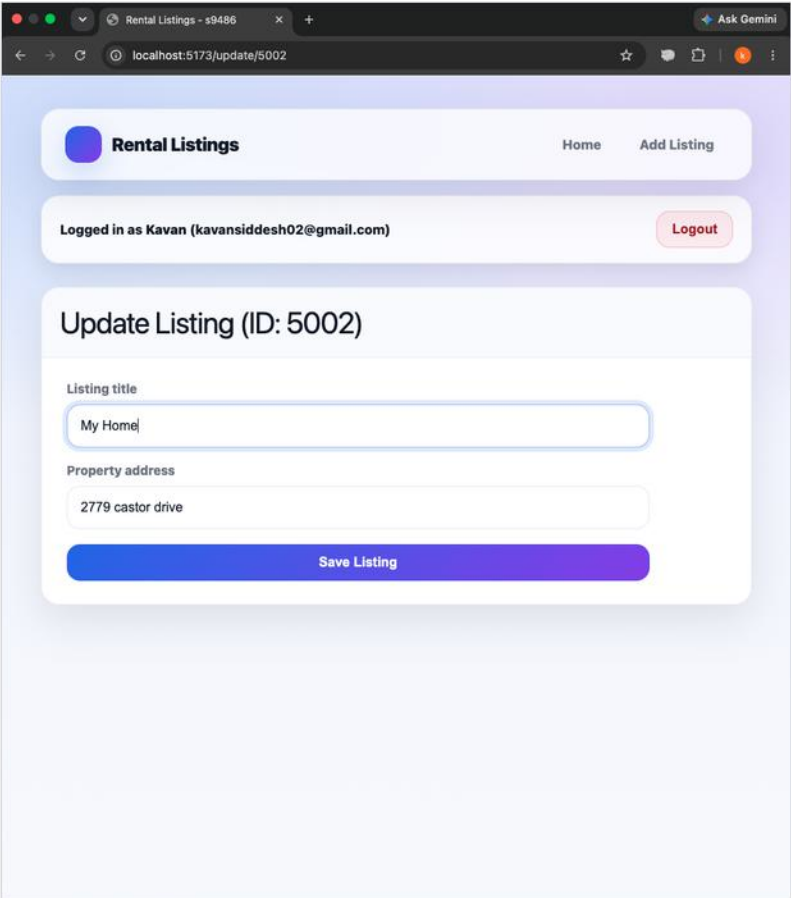
+ Add Listing

ID	TITLE	ADDRESS	ACTIONS
5002	Home	2779 castor drive	Update Delete
5000	Quiet 2BR Townhouse near downtown	9241 San Carlos St, Mountain View, CA	Update Delete
4999	Spacious 1BR Condo with parking	7340 Market St, Mountain View, CA	Update Delete
4998	Furnished 1BR Condo near downtown	8801 Alameda, Campbell, CA	Update Delete

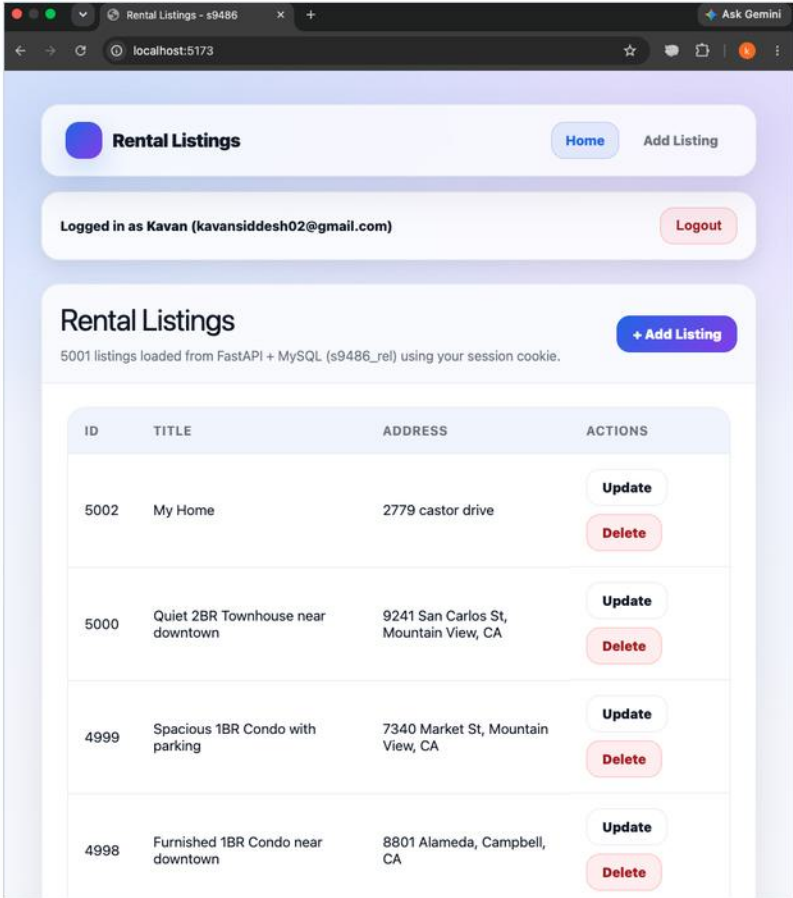
Redirected to / - new listing ID 5002 at the top

Part 1. React Client - III. Update Record (UpdateRecord.jsx, route /update/:id)

UpdateRecord reads the id with useParams, loads the listing in useEffect, and calls the onUpdate prop; the PUT is persisted in MySQL before redirecting to /.



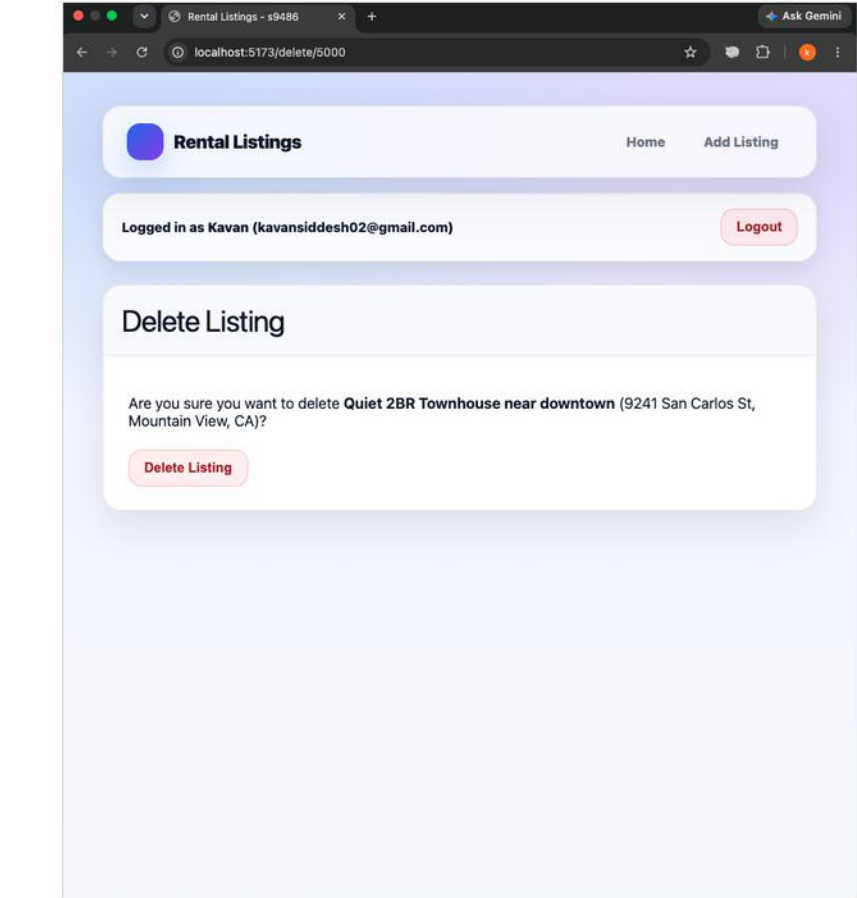
Update Listing form for ID 5002



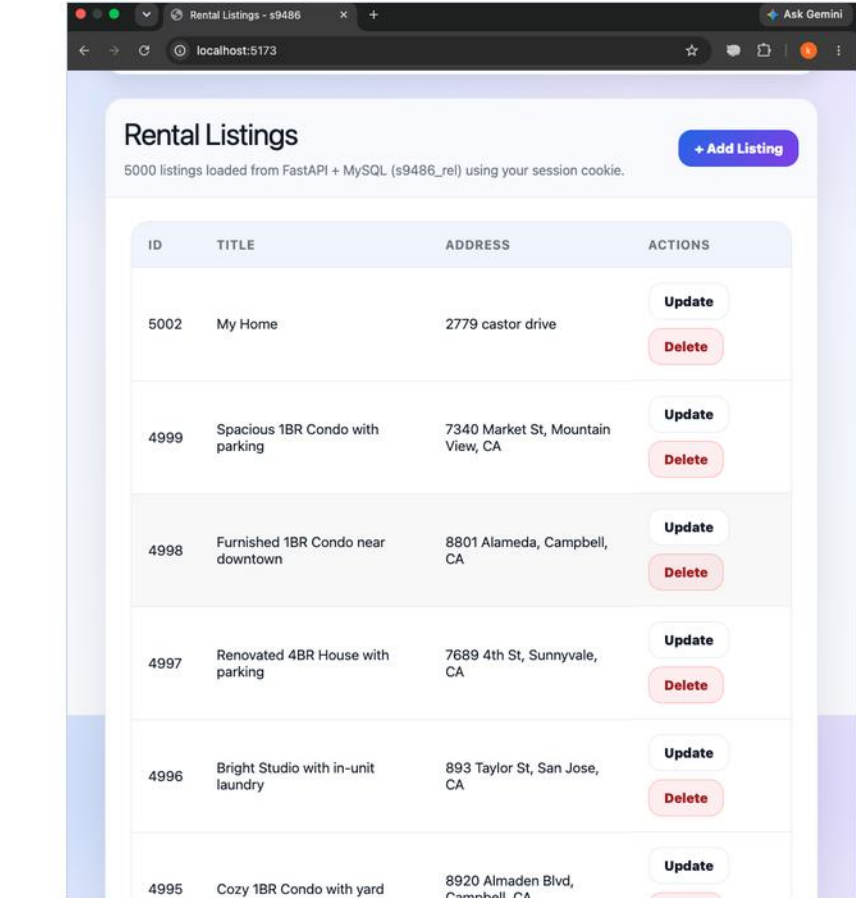
Redirected to / - ID 5002 now shows 'My Home'

Part 1. React Client - IV. Delete Record (DeleteRecord.jsx, route /delete/:id)

DeleteRecord shows the listing and a Delete Listing button; the onDelete prop sends DELETE /listings/{id}, removes the row from state and redirects to /.



Delete confirmation for listing 5000



Redirected to / - listing 5000 is gone, count back to 5,000

The connection URL comes from .env (never committed). database.py refuses to start unless the URL points at s9486_rel. scripts/init_db.py runs CREATE DATABASE IF NOT EXISTS and Base.metadata.create_all, which only creates missing tables and never drops data.

```
1  USE s9486_rel;
2  SHOW TABLES; DESCRIBE listings; DESCRIBE users; DESCRIBE sessions; DESCRIBE listing_inquiries;
3
```

Result Grid Filter Rows: Export:

	Tables_in_s9486_rel
	listing_inquiries
	listings
	sessions
	users

	Time	Action
4	13:56:15	SHOW TABLES

```

1 USE s9486_rel;
2 SHOW TABLES; DESCRIBE listings; DESCRIBE users; DESCRIBE sessions; DESCRIBE
3 
```

Result Grid

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
title	varchar(200)	NO		NULL	
address	varchar(255)	NO		NULL	

Export:

Time	Action
✓ 5 13:56:30	DESCRIBE listings

DATA 260 - Homework 4

```
1  •  USE s9486_rel;
2  SHOW TABLES; DESCRIBE listings; DESCRIBE users; DESCRIBE sessions; DESCRIBE 1
3
```

[illegible]

Action

13:56:37

DESCRIBE users

SCRIBE users - id, name, email (UNI), password_hash

```
1  *  USE s9486_rel;  
2  SHOW TABLES; DESCRIBE listings; DESCRIBE users; DESCRIBE sessions; DESCRIBE T  
3
```

[illegible]

Action

13:56:46

DESCRIBE sessions

Subscribe sessions - id (session token), user_id, created_at, expires_at

```
1  USE s9486_rel;  
2  SHOW TABLES; DESCRIBE listings; DESCRIBE users; DESCRIBE sessions; DESCRIBE  
3
```

[illegible]

Action

13:56:56


```
1  *  USE s9486_rel;
2  SHOW TABLES; DESCRIBE listings; DESCRIBE users; DESCRIBE sessions; DESCRIBE
3  *  SELECT * FROM sessions;
4
```



	id	title	address	
	5002	My Home	2779 castor drive	
	4999	Spacious 1BR Condo with parking	7340 Market St, Mountain View, CA	
	4998	Furnished 1BR Condo near downtown	8801 Alameda, Campbell, CA	
	4997	Renovated 4BR House with parking	7689 4th St, Sunnyvale, CA	
	4996	Bright Studio with in-unit laundry	893 Taylor St, San Jose, CA	
	4995	Cozy 1BR Condo with yard	8920 Almaden Blvd, Campbell, CA	
	4994	Furnished Studio pet friendly	252 Willow St, Campbell, CA	
	4993	Modern 2BR Apartment with parking	791 Almaden Blvd, Santa Clara, CA	
	4992	Spacious 2BR Apartment with yard	5218 4th St, Mountain View, CA	
	4991	Furnished 2BR Apartment near SJSU	2160 Alameda, Milpitas, CA	
	NULL	NULL	NULL	

Action

13:57:56

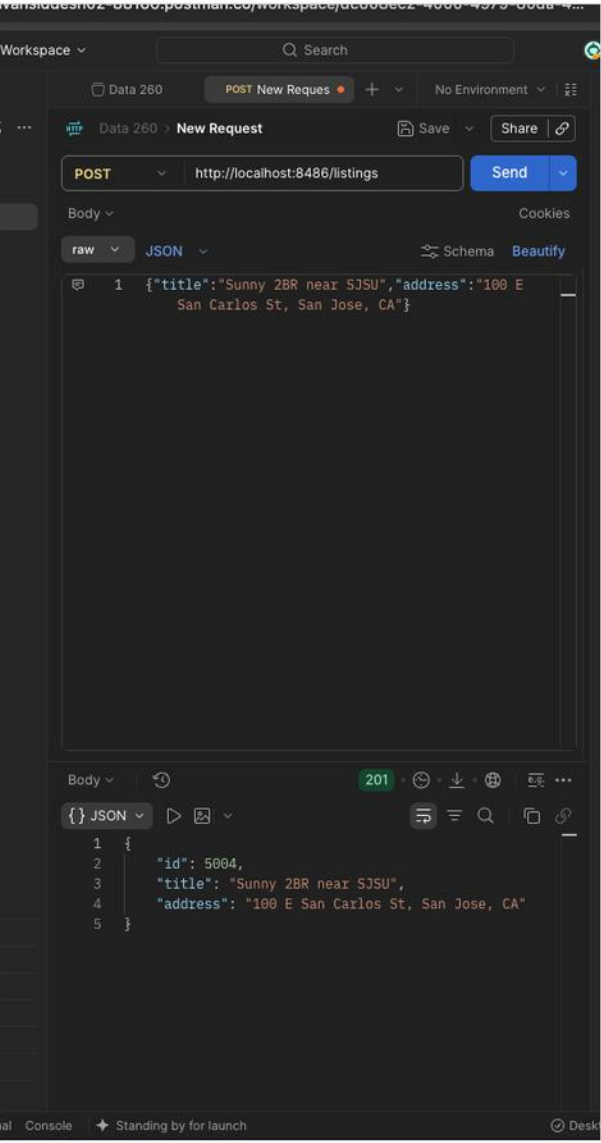
st listings - 5002 'My Home' from the React test; 5004 was deleted in Postman

Login checks the bcrypt hash, inserts a row into sessions with a 256-bit random token (`secrets.token_hex(32)`), and sends only that opaque token in an `HttpOnly, SameSite=Lax` cookie. No name or email is stored in the cookie. Every protected route looks the token up in MySQL and returns 401 when it is missing, unknown or expired.

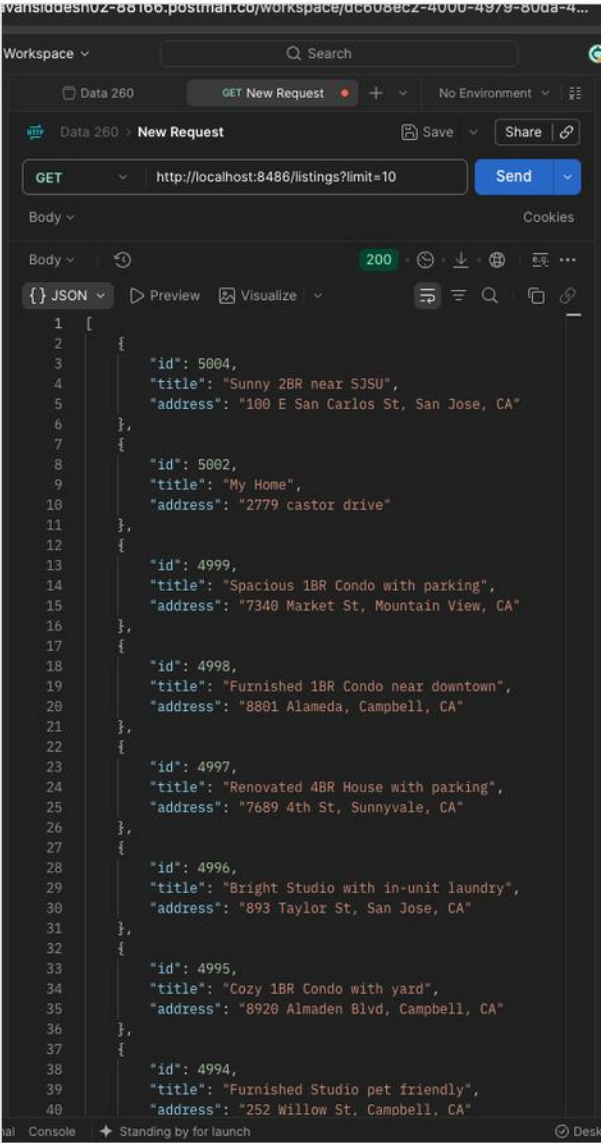


sessions table: opaque 64-character tokens linked to user id 3, expiring 30 minutes after creation

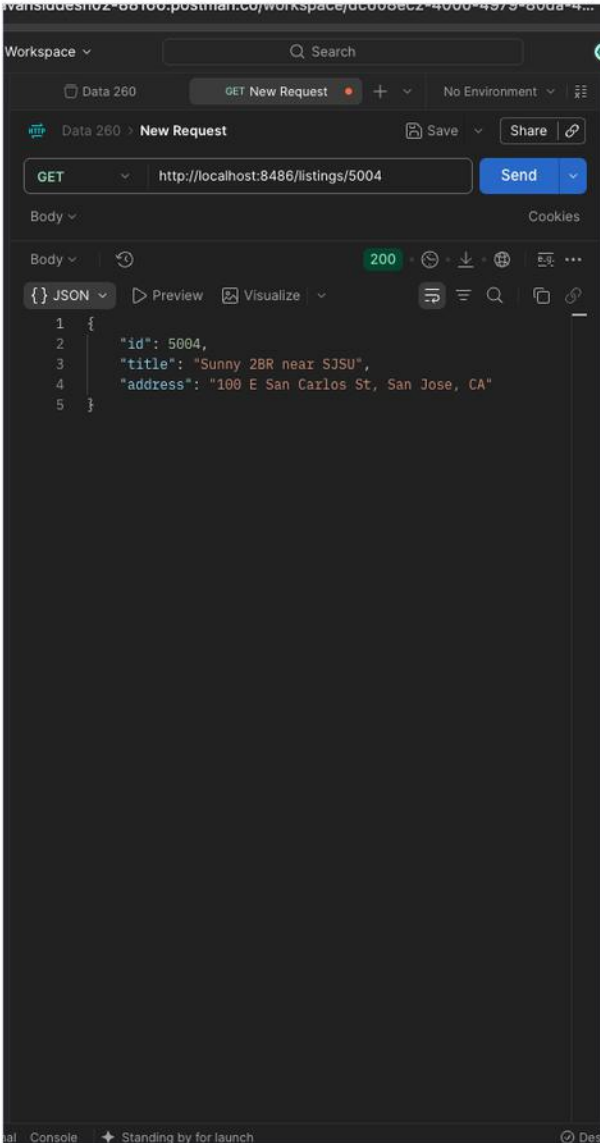
Part 2. CRUD API - Create (POST) and Read (GET all, GET by ID)



POST /listings - 201 Created, id 5004

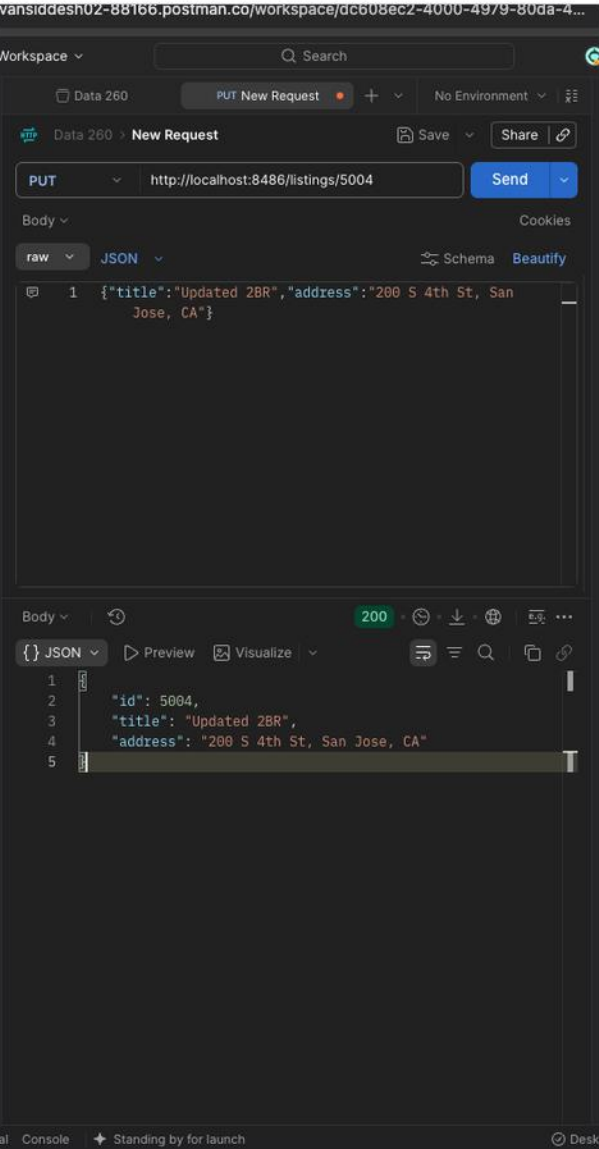


GET /listings?limit=10 - 200

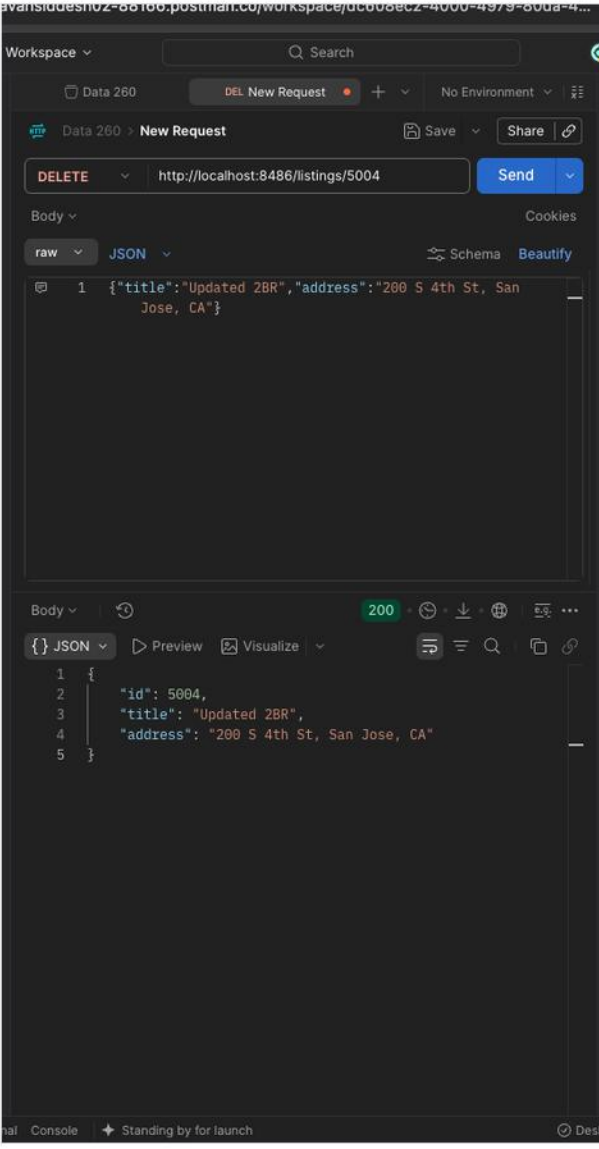


GET /listings/5004 - 200

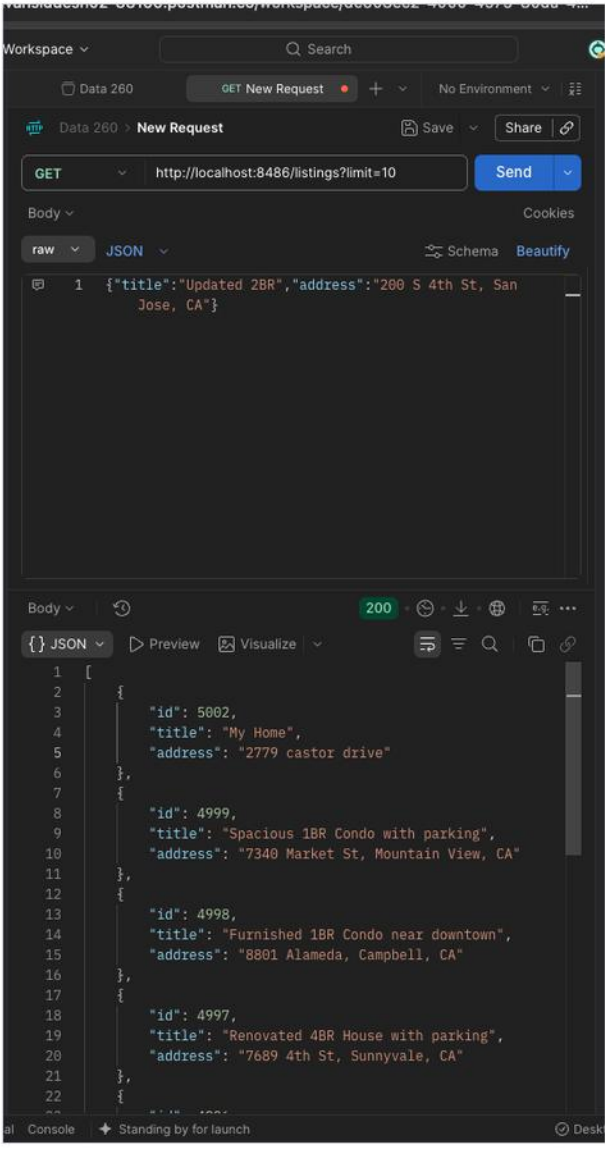
Part 2. CRUD API - Update (PUT) and Delete (DELETE)



PUT /listings/5004 - 200, updated title and address

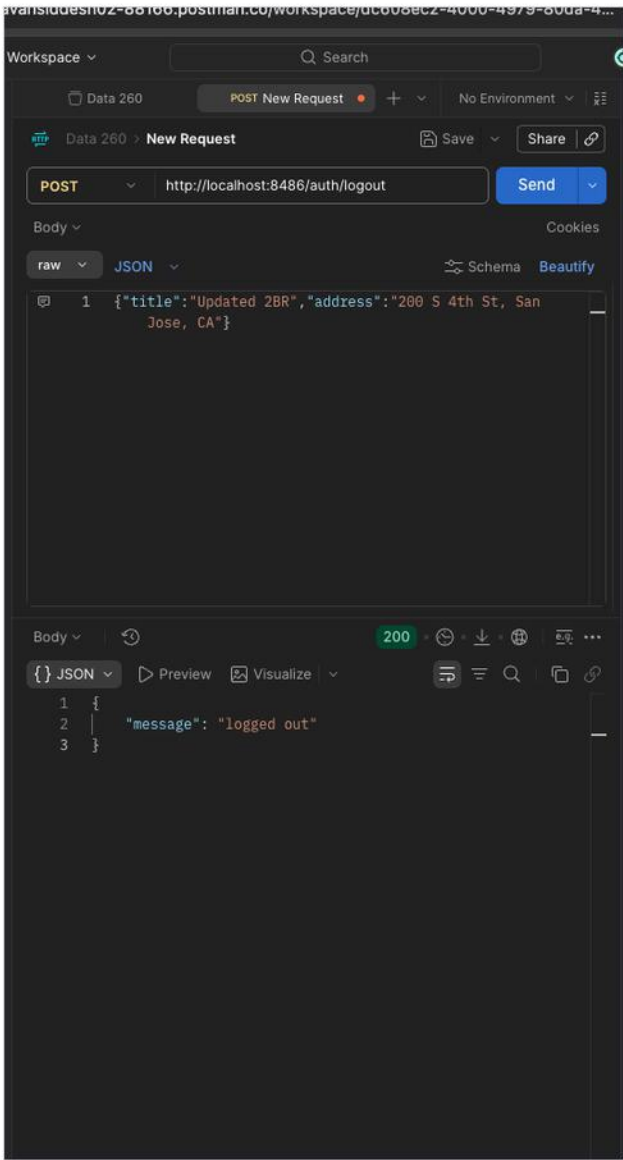


DELETE /listings/5004 - 200, deleted row returned

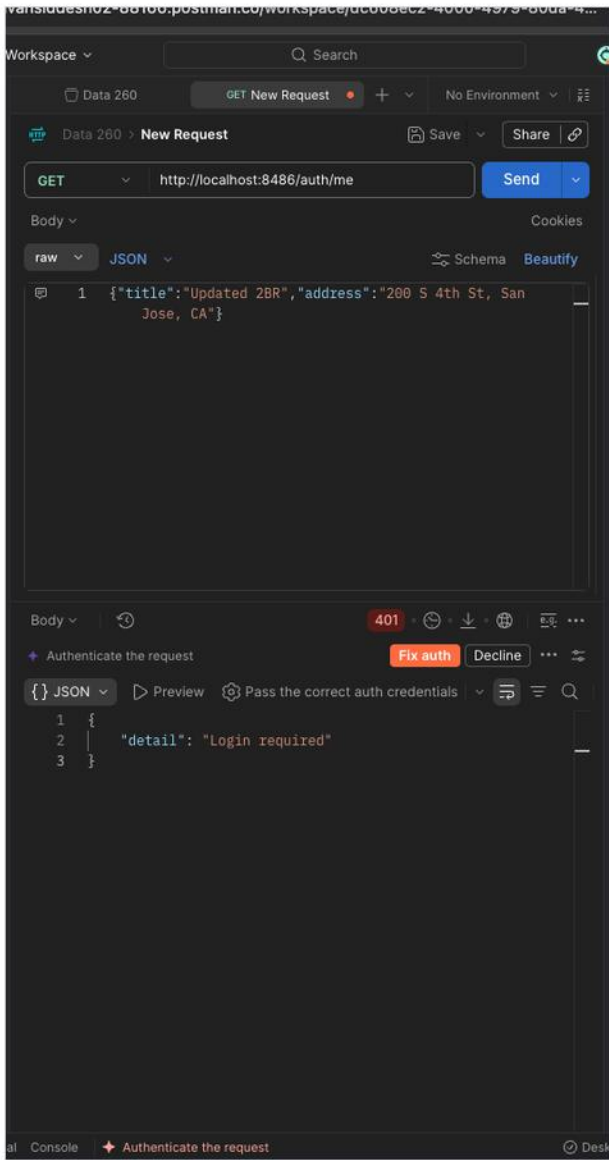


GET /listings after delete - 5004 no longer listed

Part 2. Logout - Server-Side Session Removed



POST /auth/logout - 200 logged out



GET /auth/me after logout - 401 Login required

```
Console output - make seed (2026-09-28 12:15:35, from RUN_LOG.txt)
```

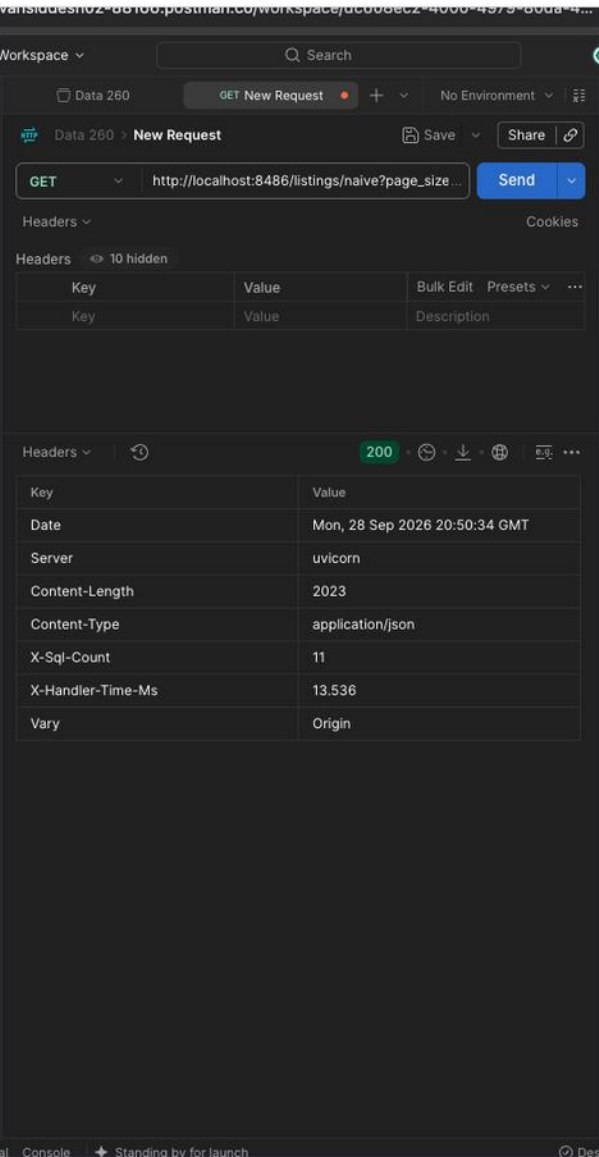
```
SEED=9486: listings=5000, listing inquiries=200, listings with >=1 inquiry=125
```

[illegible]

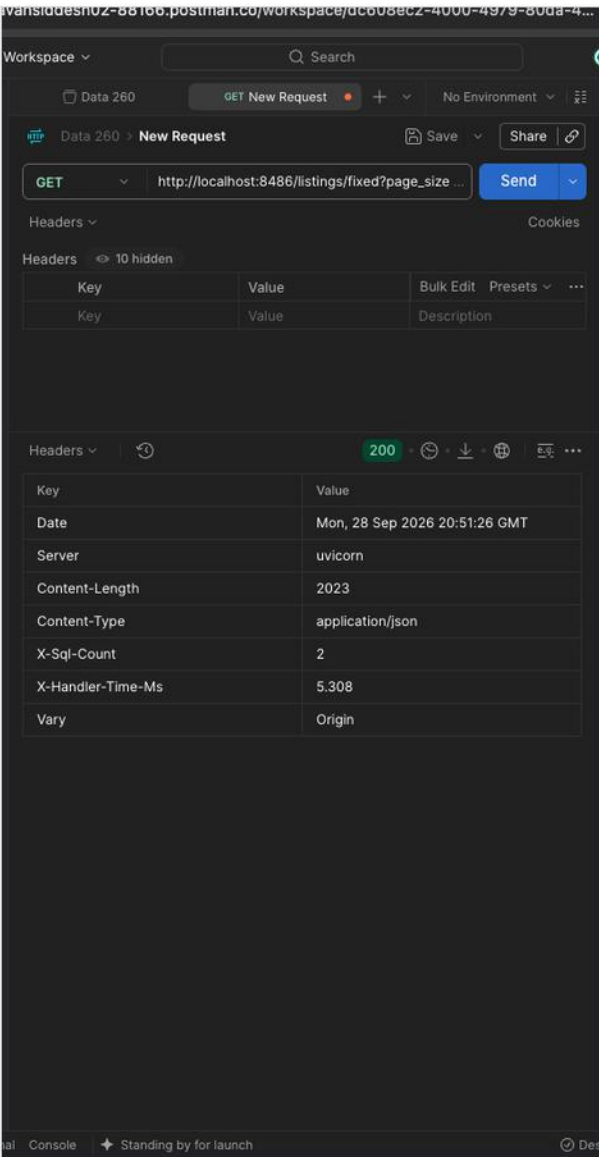
Part 3. N+1 - Naive and Fixed List Endpoints

The naive loader runs one query for the page and then one SELECT per listing for its inquiries (1 + N statements). The fixed loader uses `selectinload`, which fetches the inquiries for the whole page in a single `WHERE listing_id IN (...)` query (2 statements). A SQLAlchemy `before_cursor_execute` listener counts the statements of each request and returns the count in the X-SQL-Count header.

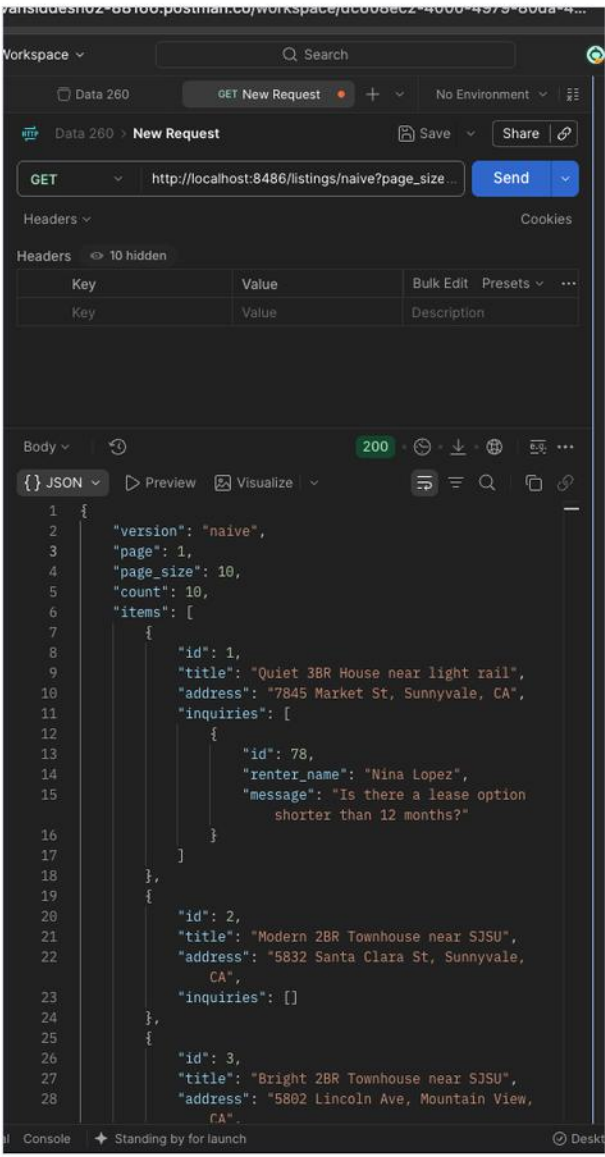
Part 3. N+1 - Postman, Both Versions at Page Size 10



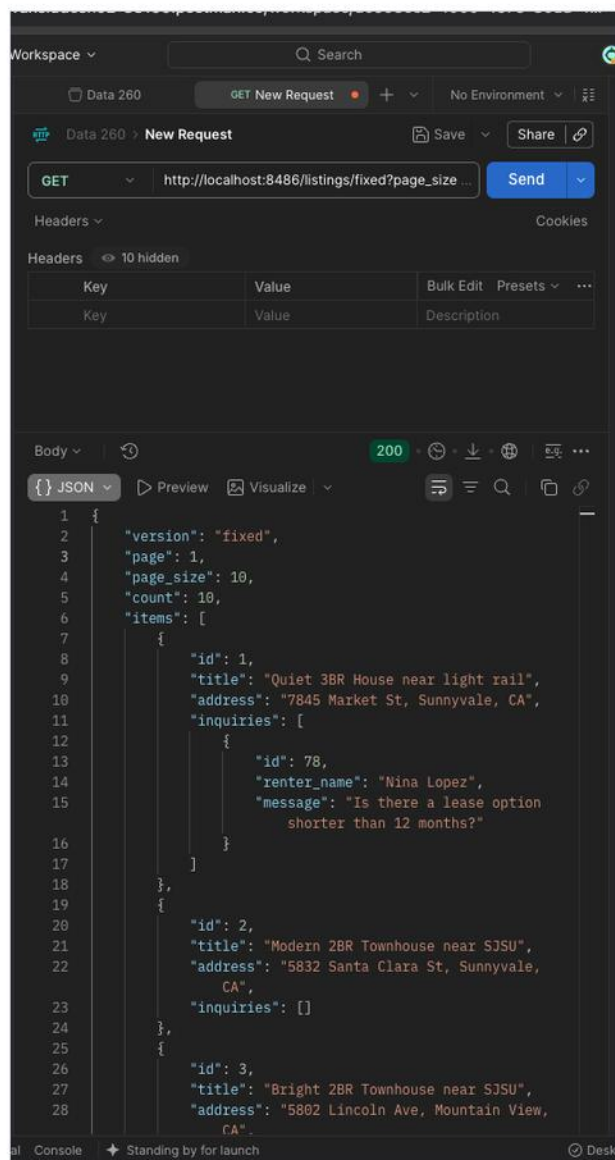
naive, page_size=10 - X-SQL-Count 11



fixed, page_size=10 - X-SQL-Count 2

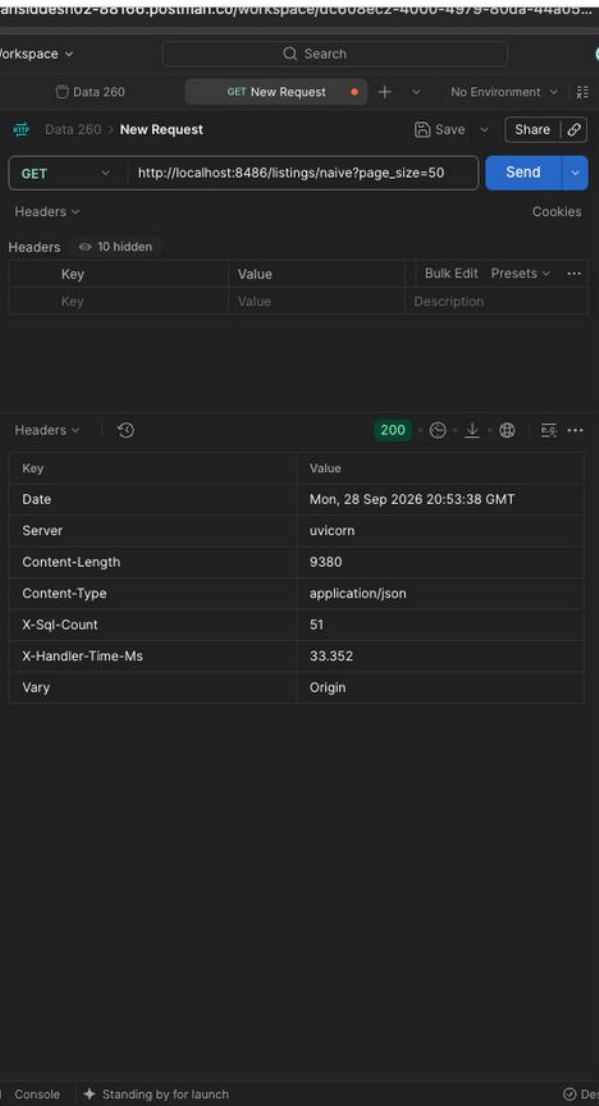


naive, page_size=10 - body with inquiries

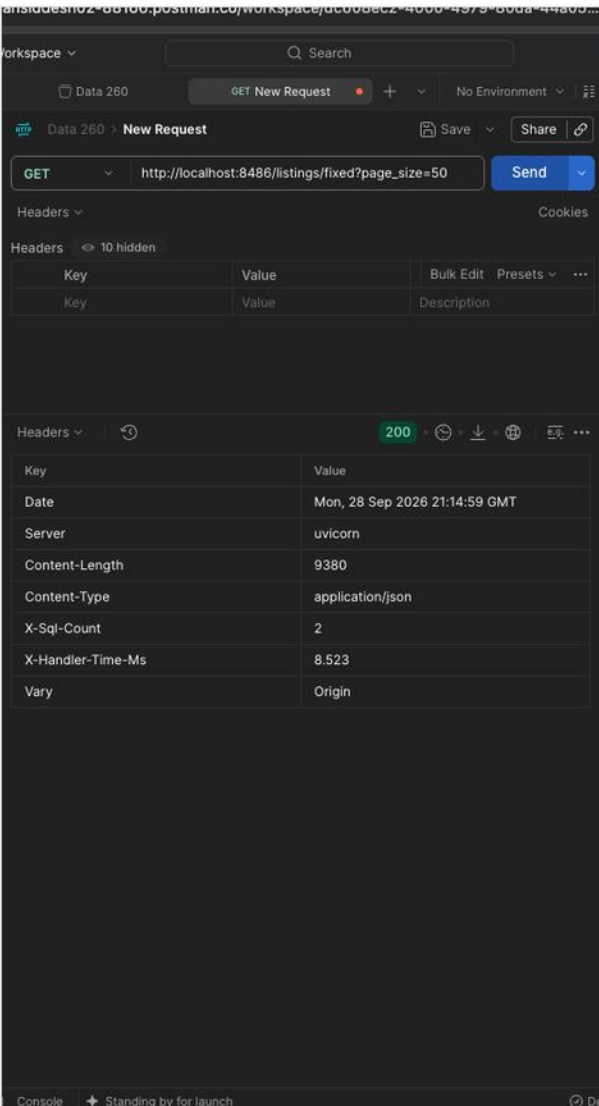


fixed, page_size=10 - same items and inquiries as naive

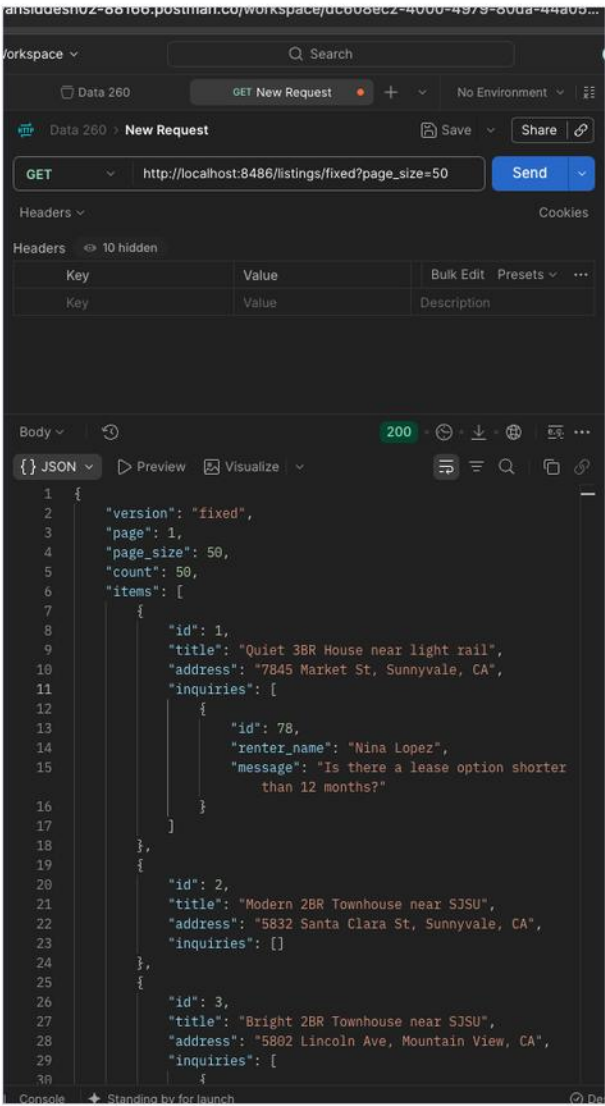
Part 3. N+1 - Postman, Both Versions at Page Size 50



naive, page_size=50 - X-SQL-Count 51

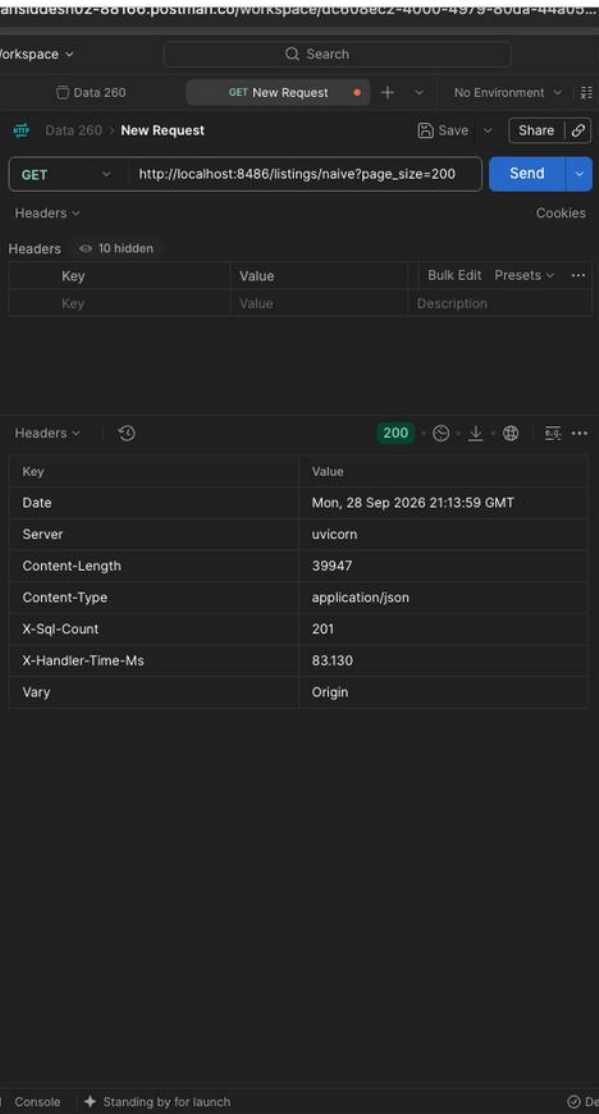


fixe, page_size=50 - X-SQL-Count 2

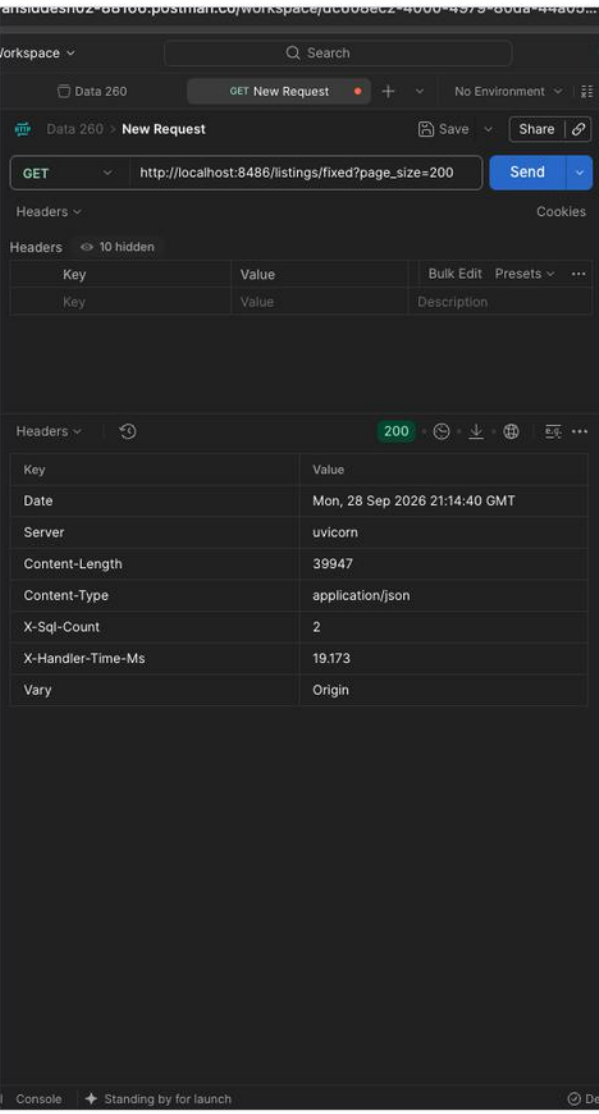


fixe, page_size=50 - body (count 50)

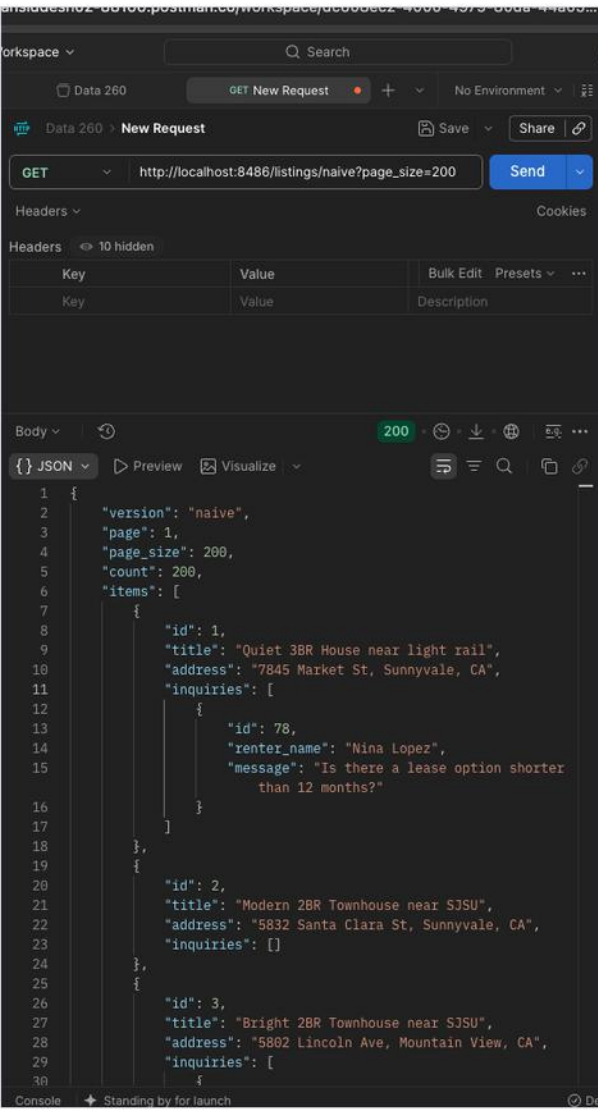
Part 3. N+1 - Postman, Both Versions at Page Size 200



naive, page_size=200 - X-SQL-Count 201



fix, page_size=200 - X-SQL-Count 2



naive, page_size=200 - body (count 200)

Part 3. N+1 - Measurement: 3 Page Sizes x 2 Versions x 30 Requests = 180

measure_n1.py logs in, sends 3 unrecorded warm-up requests per configuration, then 30 recorded requests. Latency is the client-side round trip (perf_counter); percentiles use the nearest-rank method, so with 30 samples p99 is the slowest request. All 180 rows are saved in raw/n1_requests.csv and raw/n1_requests.json.

Console output - make measure (2026-09-28 12:15:52, from RUN_LOG.txt)

```
[2026-09-28T12:15:53] N+1 measurement start - SEED=9486, sizes=[10, 50, 200], 30 req each
size=10 naive sql/req=11 items=10 inquiries=10 p50=7.90ms errors=0
size=10 fixed sql/req=2 items=10 inquiries=10 p50=4.02ms errors=0
size=50 naive sql/req=51 items=50 inquiries=44 p50=25.73ms errors=0
size=50 fixed sql/req=2 items=50 inquiries=44 p50=5.43ms errors=0
size=200 naive sql/req=201 items=200 inquiries=200 p50=92.90ms errors=0
size=200 fixed sql/req=2 items=200 inquiries=200 p50=9.75ms errors=0
... (20 more lines in RUN_LOG.txt)
```

Page size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)
10	naive	11	7.90	8.83	8.95
10	fixed	2	4.02	4.46	4.60
50	naive	51	25.73	27.04	28.16
50	fixed	2	5.43	5.94	6.08
200	naive	201	92.90	96.75	117.97
200	fixed	2	9.75	10.41	10.56

Part 3. N+1 - How Much Faster, and Why the Speedup Grows

Page size	Speedup at p50 (naive / fixed)	Speedup at p95	SQL statements saved per request
10	1.97x	1.98x	9
50	4.74x	4.55x	49
200	9.53x	9.29x	199

The naive version's statement count grows with the page (11, 51, 201) while the fixed version always runs 2 statements. Each extra naive statement is another round trip to MySQL plus parse/execute and ORM overhead, so naive latency grows almost linearly: from page size 10 to 200 its p50 went from 7.90 ms to 92.90 ms, roughly 0.45 ms per extra listing. The fixed version only pays for more rows in the same two queries and for serializing them, about 0.03 ms per extra listing (4.02 ms to 9.75 ms).

At page size 10 both versions share a fixed per-request cost (HTTP handling, session lookup, JSON encoding) of a few milliseconds, and only 9 extra queries separate them, so the speedup is just 1.97x. As the page grows, the extra queries dominate the naive time while the fixed cost stays nearly flat, so the speedup rises to 4.74x at 50 and 9.53x at 200. It keeps widening with page size because the naive cost is O(N) round trips and the fixed cost is O(1) round trips. The naive p99 at 200 (117.97 ms vs p95 96.75 ms) also shows that more round trips give more chances for an occasional slow one.

Part 3. Index and EXPLAIN Before / After

Index added on the column used by both inquiry lookups:

```
CREATE INDEX ix_listing_inquiries_listing_id ON listing_inquiries (listing_id);
```

Console output - make explain (raw/explain_before_after.txt)

```
EXPLAIN before/after - run at 2026-09-28T12:15:36

===== BEFORE (no index on listing_inquiries.listing_id) =====
-- naive_per_listing
EXPLAIN SELECT id, listing_id, renter_name, message FROM listing_inquiries WHERE listing_id = 7 ORDER BY id;
table | type | possible_keys | key | key_len | ref | rows | filtered | Extra
listing_inquiries | index | None | PRIMARY | 4 | None | 200 | 10.0 | Using where

-- fixed_selectin_page10
EXPLAIN SELECT id, listing_id, renter_name, message FROM listing_inquiries WHERE listing_id IN (1,2,3,4,5,6,7,8,9,10) ORDER BY id;
table | type | possible_keys | key | key_len | ref | rows | filtered | Extra
listing_inquiries | index | None | PRIMARY | 4 | None | 200 | 50.0 | Using where

===== INDEX ADDED =====
CREATE INDEX ix_listing_inquiries_listing_id ON listing_inquiries (listing_id);

===== AFTER =====
-- naive_per_listing
EXPLAIN SELECT id, listing_id, renter_name, message FROM listing_inquiries WHERE listing_id = 7 ORDER BY id;
table | type | possible_keys | key | key_len | ref | rows | filtered | Extra
listing_inquiries | ref | ix_listing_inquiries_listing_id | ix_listing_inquiries_listing_id | 4 | const | 1 | 100.0 | None

-- fixed_selectin_page10
EXPLAIN SELECT id, listing_id, renter_name, message FROM listing_inquiries WHERE listing_id IN (1,2,3,4,5,6,7,8,9,10) ORDER BY id;
table | type | possible_keys | key | key_len | ref | rows | filtered | Extra
listing_inquiries | range | ix_listing_inquiries_listing_id | ix_listing_inquiries_listing_id | 4 | None | 11 | 100.0 | Using index condition; Using filesort
```

Query	Before: type / key / rows	After: type / key / rows
naive per-listing lookup (listing_id = 7)	index / PRIMARY / 200 (filtered 10%, Using where)	ref / ix_listing_inquiries_listing_id / 1 (filtered 100%)
fixed selectin lookup (listing_id IN (1..10))	index / PRIMARY / 200 (filtered 50%, Using where)	range / ix_listing_inquiries_listing_id / 11 (Using index condition; Using filesort)

What changed: without an index on listing_id, MySQL had no usable key (possible_keys = NULL) and walked the whole clustered primary key (type=index, 200 rows) and filtered every row with Using where, only choosing PRIMARY to get the ORDER BY id order for free. After adding the index, the per-listing lookup became type=ref with ref=const, reading about 1 row, and the IN query became a range scan over 11 index entries. The optimizer now adds a small filesort to order those 11 rows by id, which is much cheaper than scanning all 200 rows. With only 200 related rows the latency effect is small, but the rows examined per query drop from the whole table to just the matches, and that gap grows with the table size.

Part 4. RAG - Corpus, Chunking and Index

Five public rental-housing documents (California Civil Code 1941.1, 1946, 1946.2, 1950.5 and the DOJ Fair Housing Act page) are cleaned from HTML, split into 165 chunks of 500 characters with 50 characters of overlap, embedded with all-MiniLM-L6-v2 and loaded into a FAISS IndexFlatIP (cosine similarity on normalized vectors). Characters were used instead of tokens because MiniLM truncates input at 256 tokens. Every chunk stores its text, source file name and chunk_id.

Console output - make rag (2026-09-28 12:44:06, from RUN_LOG.txt)

```
Loaded 5 documents -> 165 chunks (chunk_size=500, overlap=50 chars)
ca_civil_code_1941_1_habitability.txt: 13 chunks
ca_civil_code_1950_5_security_deposit.txt: 60 chunks
ca_tenant_notice_requirements.txt: 6 chunks
ca_tenant_protection_act_ab1482.txt: 54 chunks
fair_housing_act.txt: 32 chunks

... (219 more lines in RUN_LOG.txt)
```

Part 4. RAG - Retrieval Printed Before the LLM Call (top_k = 3)

raw/retrieved_chunks.txt - Q1 and Q2 (full file has every question and k)

```
=== Q1 (top_k=3) Under California Civil Code 1950.5, how many days after the tenant moves out does the landlord have to return the remaining security deposit with an itemized statement?
#1 score=0.6499 source=ca_civil_code_1950_5_security_deposit.txt chunk_id=ca_civil_code_1950_5_security_deposit#025
    but prior to any repairs or cleanings for which the landlord will make a deduction from or claim against the security deposit pursuant to this section, and shall also take photographs of the unit with
in a reasonable time after such repairs or cleanings are completed. (h) (1) No later than 21 calendar days after the tenant has vacated the premises, but not earlier than the time that either the landlor
d or the tenant provides a notice to terminate the tenancy under Section 1946 or 1946.1, Section
#2 score=0.6405 source=ca_civil_code_1950_5_security_deposit.txt chunk_id=ca_civil_code_1950_5_security_deposit#045
    the tenant at the same time or after a notice to terminate a tenancy under Section 1946 or 1946.1 has been given, a notice under Section 1161 of the Code of Civil Procedure has been given, or no earli
er than 60 calendar days prior to the expiration of a fixed-term lease. The waiver shall substantially include the text of paragraph (2). (5) Notwithstanding paragraph (4), the landlord shall comply with
paragraphs (2) and (3) when a tenant makes a request for documentation within 14 calendar days
#3 score=0.6346 source=ca_civil_code_1950_5_security_deposit.txt chunk_id=ca_civil_code_1950_5_security_deposit#026
    the tenancy under Section 1946 or 1946.1, Section 1161 of the Code of Civil Procedure, or not earlier than 60 calendar days prior to the expiration of a fixed-term lease, the landlord shall furnish th
e tenant, a copy of an itemized statement indicating the basis for, and the amount of, any security received and the disposition of the security, and shall return any remaining portion of the security to
the tenant as follows: (A) (i) Subject to subparagraph (C), and except as provided by clause
=== Q2 (top_k=3) Under Civil Code 1946.2, how long must a tenant have lived in the unit before the owner needs just cause to end the tenancy, and how much relocation assistance is owed for a no-fault just
cause termination?
#1 score=0.6612 source=ca_tenant_protection_act_ab1482.txt chunk_id=ca_tenant_protection_act_ab1482#030
    month of the tenancy, prior to the rent becoming due. (2) If an owner issues a notice to terminate a tenancy for no-fault just cause, the owner shall notify the tenant in the written termination notic
e of the tenant's right to relocation assistance or rent waiver pursuant to this section. If the owner elects to waive the rent for the final month of the tenancy as provided in subparagraph (B) of parag
raph (1), the notice shall state the amount of rent waived and that no rent is due for the final
#2 score=0.6591 source=ca_civil_code_1950_5_security_deposit.txt chunk_id=ca_civil_code_1950_5_security_deposit#045
    the tenant at the same time or after a notice to terminate a tenancy under Section 1946 or 1946.1 has been given, a notice under Section 1161 of the Code of Civil Procedure has been given, or no earli
er than 60 calendar days prior to the expiration of a fixed-term lease. The waiver shall substantially include the text of paragraph (2). (5) Notwithstanding paragraph (4), the landlord shall comply with
paragraphs (2) and (3) when a tenant makes a request for documentation within 14 calendar days
#3 score=0.6448 source=ca_tenant_protection_act_ab1482.txt chunk_id=ca_tenant_protection_act_ab1482#045
    law also provides that after all of the tenants have continuously and lawfully occupied the property for 12 months or more or at least one of the tenants has continuously and lawfully occupied the pro
perty for 24 months or more, a landlord must provide a statement of cause in any notice to terminate a tenancy. See Section 1946.2 of the Civil Code for more information." The notification or lease provi
sion shall be subject to Section 1632. (g) An owner's failure to comply with any provision of
```

Part 4. RAG - Three Configurations

(A) No RAG sends only the question. (B) Basic RAG pastes the top-3 raw chunks. (C) Context-engineered RAG drops chunks under a cosine score of 0.35 and near-duplicates (word Jaccard >= 0.8), groups the survivors by source and labels them [1], [2], ..., and adds grounding rules with source-number citations and the exact refusal sentence. If no chunk survives, it refuses without calling the model.

Part 4. RAG - Six-Question Test Set and Results (llama3.2:3b, top_k = 3)

ID	Type	Question	Expected answer
Q1	answer in one chunk	Under California Civil Code 1950.5, how many days after the tenant moves out does the landlord have to return the remaining security deposit with an itemized statement?	No later than 21 calendar days after the tenant has vacated the premises.
Q2	answer needs two chunks	Under Civil Code 1946.2, how long must a tenant have lived in the unit before the owner needs just cause to end the tenancy, and how much relocation assistance is owed for a no-fault just cause termination?	12 months of continuous and lawful occupancy; relocation assistance (or a rent waiver) equal to one month of the tenant's rent.
Q3	similar information across documents	How many days of written notice are required to end a month-to-month tenancy?	At least 30 days' written notice (Civil Code 1946). The habitability document also mentions 30 days' written notice, but for returning a tenant-owned refrigerator, not for ending a tenancy.
Q4	ambiguous	How much can the landlord charge?	Ambiguous. A good answer asks what kind of charge is meant or states its assumption; the clearest match is the security deposit cap of one month's rent in Civil Code 1950.5(c).
Q5	answer not in the documents	What is the maximum late fee a California landlord can charge when rent is paid late?	I cannot answer this question from the provided documents
Q6	unrelated	What is the capital city of Australia?	I cannot answer this question from the provided documents

Q	No RAG	Basic RAG	Context RAG
Q1	Within 30 days after the tenant moves out.	According to the context, the landlord must return the remaining security deposit with an itemized statement within 21 calendar days after the tenant has vacated the premises.	According to [1], the landlord must return the remaining security deposit with an itemized statement no later than 21 calendar days after the tenant has vacated the premises.
Q2	I can't provide information on the Civil Code of 1946. Can I help you with something else?	According to the context, under Civil Code 1946.2, a landlord must provide a statement of cause in any notice to terminate a tenancy if: * All tenants have continuously and lawfully occupied the property for 12 months or more. * At least one of the tenants has continuously and lawfully occupied the property for 24 months or more. There is no specific mention of the amount of relocation assistance owed for a no-fault just cause termination.	I cannot answer this question from the provided documents
Q3	Typically, 30 days of written notice is required to end a month-to-month tenancy.	The context does not explicitly state the number of days of written notice required to end a month-to-month tenancy. However, it does state that parties may terminate the same by giving at least 30 days' written notice thereof at any time.	According to [1], at least 30 days' written notice is required to terminate a tenancy, but it does not specify whether this applies to month-to-month tenancies.
Q4	The amount a landlord can charge varies depending on the location, type of property, and local laws. Typically, landlords can charge: * Rent: 1-3 times the monthly market rate * Security deposit: 1-2 months' rent * Late fees: \$10-\$50 per day * Pet fees: \$20-\$100 per month * Utility fees: varies It's essential to review local laws and regulations to determine the maximum allowed charges.	According to the context, the landlord can charge a security amount equal to two months' rent, in addition to any rent for the first month paid on or before initial occupancy.	According to section (5) [1], a landlord shall not demand or receive security in an amount or value in excess of two months' rent, in addition to any rent for the first month paid on or before initial occupancy.
Q5	In California, the maximum late fee a landlord can charge is \$10 per month, or 5% of the unpaid rent, whichever is less.	The context does not explicitly state the maximum late fee a California landlord can charge when rent is paid late. However, it does mention that California law limits the amount a landlord can increase rent, as stated in Section 1947.12 of the Civil Code.	I cannot answer this question from the provided documents
Q6	Canberra.	I don't have enough information to answer your question about the capital city of Australia. The context appears to be related to the FEAR Act Data and accommodations for persons with disabilities, and does not provide any information about Australia or its capital city. If you could provide more context or clarify the question, I'd be happy to try and assist you further.	I cannot answer this question from the provided documents

Part 4. RAG - Refusals on Q5 and Q6

Q	Config	Answer	Exact refusal	Refused by
Q5	no_rag	In California, the maximum late fee a landlord can charge is \$10 per month, or 5% of the unpaid rent, whichever is less.	no	-
Q6	no_rag	Canberra.	no	-
Q5	basic_rag	The context does not explicitly state the maximum late fee a California landlord can charge when rent is paid late. However, it does mention that California law limits the amount a landlord can increase rent, as stated in Section 1947.12 of the Civil Code.	no	-
Q6	basic_rag	I don't have enough information to answer your question about the capital city of Australia. The context appears to be related to the FEAR Act Data and accommodations for persons with disabilities, and does not provide any information about Australia or its capital city. If you could provide more context or clarify the question, I'd be happy to try and assist you further.	no	-
Q5	context_rag	I cannot answer this question from the provided documents	yes	model
Q6	context_rag	I cannot answer this question from the provided documents	yes	retrieval gate

Only Context RAG returned the exact sentence for both questions. Q6 was refused by the retrieval gate (best score 0.10), and Q5 passed the gate (scores about 0.49) but the model refused under the grounding rule. No RAG invented a late-fee rule for Q5 and answered Q6 from general knowledge; Basic RAG declined in its own words, which does not meet the required format.

Part 4. RAG - Context-Size Sweep (k = 1, 3, 5)

Q	k	Retrieved (source, score)	Dropped	Correct answer	Answer
Q1	1	ca_civil_code_1950_5_security_deposit#025 (0.65)	-	True	According to California Civil Code 1950.5, the landlord must return the remaining security deposit with an itemized statement no later than 21 calendar days after the tenant has vacated the premises.
Q1	3	ca_civil_code_1950_5_security_deposit#025 (0.65); ca_civil_code_1950_5_security_deposit#045 (0.64); ca_civil_code_1950_5_security_deposit#026 (0.63)	-	True	According to [1], the landlord must return the remaining security deposit with an itemized statement no later than 21 calendar days after the tenant has vacated the premises.
Q1	5	ca_civil_code_1950_5_security_deposit#025 (0.65); ca_civil_code_1950_5_security_deposit#045 (0.64); ca_civil_code_1950_5_security_deposit#026 (0.63); ca_civil_code_1950_5_security_deposit#046 (0.63); ca_civil_code_1950_5_security_deposit#032 (0.57)	-	True	According to [1], the landlord shall return the remaining portion of the security to the tenant no later than 21 calendar days after the tenant has vacated the premises.
Q2	1	ca_tenant_protection_act_ab1482#030 (0.66)	-	False	I cannot answer this question from the provided documents
Q2	3	ca_tenant_protection_act_ab1482#030 (0.66); ca_civil_code_1950_5_security_deposit#045 (0.66); ca_tenant_protection_act_ab1482#045 (0.64)	-	False	I cannot answer this question from the provided documents
Q2	5	ca_tenant_protection_act_ab1482#030 (0.66); ca_civil_code_1950_5_security_deposit#045 (0.66); ca_tenant_protection_act_ab1482#045 (0.64); ca_tenant_protection_act_ab1482#031 (0.63); ca_tenant_protection_act_ab1482#029 (0.62)	-	True	According to [4], a landlord must provide a statement of cause in any notice to terminate a tenancy after all tenants have continuously and lawfully occupied the property for 12 months or more, or at least one tenant has continuously and lawfully occupied the property for 24 months or more. The amount of relocation assistance for a no-fault just cause termination is equal to one month of the tenant's rent that was in effect when the owner issued the notice to terminate the tenancy, as stated in [3].
Q3	1	ca_tenant_notice_requirements#002 (0.73)	-	True	According to the source, for a month-to-month tenancy, written notice of termination is required at least 30 days prior to the termination date.
Q3	3	ca_tenant_notice_requirements#002 (0.73); ca_tenant_protection_act_ab1482#045 (0.70); ca_civil_code_1950_5_security_deposit#045 (0.68)	-	True	According to [1], at least 30 days' written notice is required to terminate a tenancy, but it does not specify whether this applies to month-to-month tenancies.
Q3	5	ca_tenant_notice_requirements#002 (0.73); ca_tenant_protection_act_ab1482#045 (0.70); ca_civil_code_1950_5_security_deposit#045 (0.68); ca_tenant_protection_act_ab1482#030 (0.68); ca_tenant_protection_act_ab1482#001 (0.63)	-	False	I cannot answer this question from the provided documents

Q	k = 1	k = 3	k = 5	Best k
Q1	correct	correct	correct	1 (same answer, shortest context)
Q2	refused (evidence missing)	refused (evidence missing)	correct, cites [4] and [3]	5
Q3	correct	correct, hedged	refused (extra just-cause chunks confused it)	1

Part 4. RAG - Evaluation Table

Q	Config	Correct retrieval	Correct answer	Grounded	Refused when needed
Q1	no_rag	n/a	no	n/a	yes
Q1	basic_rag	yes	yes	yes	yes
Q1	context_rag	yes	yes	yes	yes
Q2	no_rag	n/a	no	n/a	yes
Q2	basic_rag	no	no	no	yes
Q2	context_rag	no	no	yes	no
Q3	no_rag	n/a	yes	n/a	yes
Q3	basic_rag	no	yes	yes	yes
Q3	context_rag	no	yes	yes	yes
Q4	no_rag	n/a	no	n/a	yes
Q4	basic_rag	no	no	no	yes
Q4	context_rag	no	no	no	yes
Q5	no_rag	n/a	no	n/a	no
Q5	basic_rag	n/a (no relevant chunk exists)	no	no	no
Q5	context_rag	n/a (no relevant chunk exists)	yes	yes	yes
Q6	no_rag	n/a	no	n/a	no
Q6	basic_rag	n/a (no relevant chunk exists)	no	no	no
Q6	context_rag	n/a (no relevant chunk exists)	yes	yes	yes

Config	Accuracy (correct answers)	Faithfulness (grounded)	Format compliance	Robustness (refusal behaviour)
no_rag	1/6	n/a	n/a	4/6
basic_rag	2/6	2/6	n/a	4/6
context_rag	4/6	5/6	6/6	5/6

Manual review of the automatic checks

Q	Config	Automatic	Manual	Reason
Q3	basic_rag, context_rag	correct retrieval = no	yes	chunk ca_tenant_notice_requirements#002 contains "at least 30 days' written notice"; the "month to month" keyword sits in the preceding chunk, so the keyword check was too strict
Q4	basic_rag, context_rag	grounded = no	yes (grounded), no (correct)	"two months' rent" is stated in the retrieved chunk, but it is the small-landlord exception in 1950.5(c)(5); neither config flagged the ambiguity or gave the general one-month rule
Q2	context_rag	refused when needed = no	refusal was faithful but incomplete	at k = 3 the relocation-amount chunk (#031) was not retrieved, so refusing was consistent with the evidence given; at k = 5 it answered both parts correctly

Part 4. RAG - Analysis

Retrieval. With top_k = 3, retrieval found the answering chunk for Q1 (1950.5 chunk #025, "21 calendar days", ranked first) and for Q3 (1946 chunk #002, "30 days' written notice", score 0.73). The 1941.1 refrigerator clause that also mentions "30 days written notice" never reached the top 5, so the cross-document distractor did not cause a problem. Q2 was only half retrieved: the just-cause chunks came back, but the chunk with the relocation amount (#031, "one month of the tenant's rent") ranked fourth, outside k = 3. For Q4 all three chunks came from 1950.5, but they described the two-month small-landlord exception instead of the general one-month rule. Q5 retrieved loosely related rent chunks (scores about 0.49), and Q6 retrieved nothing relevant (scores about 0.10).

Which context changes helped. The biggest gains came from the grounding rules and the relevance filter. No-RAG hallucinated on three questions: "30 days" for Q1, an invented "\$10 per month or 5%" late-fee rule for Q5, and "Canberra" for Q6, which is true but not grounded. Basic RAG fixed Q1 but never used the exact refusal sentence. Context RAG got Q1, Q3, Q5 and Q6 right, cited source numbers in every answer (6/6 format compliance), and refused Q5 and Q6 with the exact text. Q6 was refused by the 0.35 relevance gate before the model was called. Q5 passed the gate and was refused by the model itself because of the grounding rule. De-duplication never triggered in this run: the character windows with a 50-character overlap did not produce near-copies in the top-k. Source labels and ordering made the citations checkable, for example "[1]" in Q1 pointing to chunk #025.

Effect of k. More context was not always better. For Q2, k = 1 and k = 3 led to a refusal because the second fact was missing, while k = 5 brought in #031 and the model answered both parts with citations. For Q3, k = 1 gave the cleanest answer, k = 3 added a hedge, and k = 5 added two just-cause chunks that pushed the model into refusing. For Q1 every k worked. So the best k depended on the question: small k for single-chunk facts and larger k for multi-chunk answers.

Ambiguity and failures. Q4 was not handled well. Both RAG configs stated "two months' rent" as the limit without saying it is an exception or asking what kind of charge was meant. The answer was grounded in the retrieved text but misleading. Context RAG's refusal on Q2 at k = 3 was faithful but unhelpful.

Overall. Retrieval quality set the ceiling: when the right chunk was missing (Q2 at k = 3, Q4), no prompt could fix the answer. Context quality decided whether the model stayed on topic, and the grounding prompt turned weak evidence into refusals instead of hallucinations. Context RAG was the only configuration that never hallucinated. Its remaining errors were cautious refusals and an unflagged ambiguity, not invented facts.

1. What I used an AI assistant for, and what I did myself. I used Claude Code for a real share of the code, not only the write-up. It adapted the professor's DATA236_demo4 starter to my rental-listing domain (React pages, API helper, FastAPI routes), replaced the starter's user_id-only login with email/password login backed by bcrypt hashes and a server-side sessions table, and drafted the N+1 seed/measurement scripts, the EXPLAIN script, rag.py, and the smoke test. It also helped me improve error handling, check my work against the assignment requirements, structure the N+1 experiment, and organize and refine the RAG analysis and the reports. What I did myself: set up MySQL and the s9486_rel database, created my login user, ran every script in my own terminal (seeding, EXPLAIN, the 180 measured requests, the RAG runs with llama3.2:3b), tested the React app and every endpoint in Postman, captured all screenshots, chose and checked the corpus documents, and reviewed the numbers and model answers before putting them in METRICS.md and the report.
2. One AI-produced output that was wrong/unsuitable. The first version of rag.py called Ollama with "think": false and num_predict: 300, assuming that flag would turn off qwen3:4b's reasoning mode. It did not. In the first full RAG run every answer began with text like "Okay, the user is asking about California Civil Code 1950.5..." and was cut off at the 300-token limit before the actual answer. That made all three configurations useless for evaluation. A second, smaller issue: the AI-written keyword check marked Q3 retrieval as wrong even though the retrieved chunk contained the "30 days' written notice" answer.
3. How I detected the problem. I read the answers in RUN_LOG.txt instead of just checking that the run finished. Then I tested the model directly with a trivial prompt ("What is 2+2? Answer in one sentence.") through the Ollama API. With think: false, the /no_think switch, an empty <think></think> prefill, and a raw qwen3 chat template, the reply was always several sentences of reasoning. Only think: true separated the reasoning from the answer, and that took about 55 seconds even for 2+2, which is too slow for 27 calls on an 8 GB M1. For the Q3 check, I compared the flagged chunk text in retrieved_chunks.txt with the expected answer by hand.
4. What I changed, and why it works now. I removed qwen3:4b and switched Part 4 to llama3.2:3b, a smaller non-thinking Llama model (this also matches the "smaller Llama model" requirement). I also removed the think flag from rag.py. A single test query then returned a short, cited answer in about 13 seconds, and the full run finished in a few minutes with clean answers. For the keyword check, I kept the automatic results unchanged in raw/ and recorded the manual corrections separately in METRICS.md, so the raw output still matches what the script actually produced.

Closing Notes

Self-check: make verify-hw04 (scripts/verify_hw04.py) writes verification.json. Last run: 2026-09-28T14:21:06.

Check	Passed	Details
required_files	yes	all required files present
python_compiles	yes	17 files compiled
config_values	yes	PORT_BASE=8486, DB_NAME=s9486_rel, DOMAIN_ID=6
db_session_variable_name	yes	SessionLocal defined in database.py
backend_responds_on_port_base	yes	GET http://localhost:8486/health -> 200
create_temp_user	yes	temporary user verify-269486@example.com
unauthenticated_requests_rejected	yes	status codes without cookie: [401, 401, 401, 401, 401]
wrong_password_rejected	yes	-> 401
login_works	yes	-> 200
session_cookie_http_only_and_opaque	yes	HttpOnly=True, token is 64 hex chars=True
session_stored_server_side	yes	token found in sessions table
authenticated_requests_work	yes	/auth/me -> 200, /listings -> 200
crud_create_read_update_delete	yes	POST 201, GET 200, PUT 200, DELETE 200, GET after delete 404, invalid POST 422
naive_endpoint_returns_data	yes	items=10, includes inquiries=True, X-SQL-Count=11
fixed_endpoint_returns_data	yes	items=10, includes inquiries=True, X-SQL-Count=2
naive_and_fixed_return_same_data	yes	identical items for page 1, size 10
logout_invalidates_session	yes	/auth/me with the old token after logout -> 401
n1_raw_data_180_requests	yes	180 recorded requests
rag_raw_outputs_present	yes	all RAG outputs present

- AI use disclosure: reports/hw04/AI_USE.md
- Run log with timestamps: reports/hw04/RUN_LOG.txt
- N+1 raw data (180 requests): reports/hw04/raw/n1_requests.csv, n1_requests.json, n1_summary.json
- EXPLAIN output: reports/hw04/raw/explain_before_after.txt / .json
- RAG outputs: reports/hw04/raw/retrieved_chunks.txt, no_rag.jsonl, basic_rag.jsonl, context_rag.jsonl, six_question_results.md, refusals.json, k_sweep.jsonl / .md, evaluation_table.md, evaluation.csv
- Filled-in tables: reports/hw04/METRICS.md
- GitHub repository: <https://github.com/Kavan1804/data260-9486> (tag hw4)